Detecting Undocumented Non-Functional Dependencies in Bloated Linux Kernels Using Static Analysis

Interim Report

Student Name: E.A.D.D. Anjana

Index No: 22000135

Registration No: 2022/CS/013

Supervisor: Dr. Chamath Keppitiyagama

Co-Supervisor: Dr. Tharindu Wijethilake

Declaration

I declare that this interim report is my own original work carried out for the SCS4224 Final Year Research Project at the University of Colombo School of Computing. All sources consulted have been duly acknowledged in accordance with the University's academic integrity policy.

.....	.....
Mr. E.A.D.D. Anjana	**Date**
.....	.....
Dr. Chamath Keppitiyagama Supervisor	**Date**
.....	.....
Dr. Tharindu Wijethilake Co-Supervisor	**Date**

Contents

List of Acronyms **vi**

1 Introduction, Motivation, Aims and Objectives **1**
1.1 Introduction . . . 1
1.2 Motivation . . . 1
1.3 Revised Aims and Objectives . . . 2
1.4 Changes Since the Proposal . . . 3

2 Literature Review and Research Gap **4**
2.1 Kernel Debloating: State of the Art . . . 4
2.2 Configuration Dependencies and Static Analysis . . . 4
2.3 Identified Research Gap . . . 5
2.4 The Dependency Pattern . . . 6

3 Research Questions, Significance and Scope **7**
3.1 Revised Research Questions . . . 7
3.2 Justification of Significance . . . 7
3.3 Scope and Delimitations . . . 8

4 Research Approach and Methodology **9**
4.1 Overview . . . 9
4.2 Why Object-File Analysis Precedes Clang . . . 9
4.3 Stage 1 - Extraction . . . 10
4.4 Stage 2 - Filtering . . . 10
4.4.1 F0 - The candidate population . . . 11
4.4.2 F1 - Variables that are written . . . 11
4.4.3 F2 - Variables that are read in the subsystem of interest . . . 12
4.4.4 F3 - Separable compilation units . . . 12
4.4.5 F4 - The silent-failure filter . . . 12
4.4.6 F5 - Differing build conditions . . . 13
4.4.7 What the filters establish, and what they do not . . . 13
4.5 Stage 3 - Guard Classification (designed) . . . 14
4.6 Stage 4 - Build-Time Safety Check (designed) . . . 14

5 Experiments and Preliminary Results **15**

5.1 Experimental Setup 15
5.2 Extraction and Filtering 15
5.3 Independence from a Particular Configuration 16
5.4 Status of the Candidate Set 17
5.5 Limitations of the Present Results 17

6 Evaluation Plan **18**
6.1 What Is Claimed, and What Is Not 18
6.2 Confirmation Procedure 18
6.3 Evidence, Ranked by Independence 19
6.4 Capability Check on Known Instances 19
6.5 Comparison with Existing Analyses 19
6.6 Functional Check on the Retained Kernel 20
6.7 Evaluation of RQ2 20
6.8 Threats to Validity 20

7 Availability of Required Resources **21**

8 Project Progress and Remaining Plan **22**
8.1 Project Schedule 22
8.2 Progress to Date 23
8.3 Remaining Plan 24
8.4 Risk Assessment 25

References **26**

List of Figures

8.1 Project schedule. Dashed line: end of the interim report. Dark bars: work completed up to the interim report. Light bars: planned work. . . 22

List of Tables

1.1 Panel feedback and the resulting changes. 3

3.1 Delimitations with justifications. 8

4.1 Analysis stages and their status. 9
4.2 Filter stages. 11

5.1 Filter results for `net` readers. F0–F2 count variables; F3–F5 count writer/reader pairs. 16

6.1 Forms of confirming evidence, strongest first. 19

7.1 Resources in use. 21

8.1 Work completed. 23
8.2 Remaining work. 24
8.3 Project risks and mitigation strategies. 25

List of Acronyms

Acronym	Expansion
AST	Abstract Syntax Tree
CPU	Central Processing Unit
ELF	Executable and Linkable Format
FYP	Final Year Project
GB	Gigabyte
GCC	GNU Compiler Collection
IPv6	Internet Protocol version 6
LLVM	Low Level Virtual Machine (compiler infrastructure)
LTP	Linux Test Project
OS	Operating System
QEMU	Quick Emulator
RAM	Random Access Memory
RQ	Research Question

Chapter 1

Introduction, Motivation, Aims and Objectives

1.1. Introduction

The Linux kernel is the core software layer between applications and hardware in the vast majority of cloud servers, mobile devices, and embedded systems worldwide. Decades of growth have caused the kernel to exceed 36 million lines of code with over 19,000 configuration options (Gunasekaran et al., 2025), yet most individual deployments use only 5-10% of this code. A cloud server running a web application has no need for Bluetooth drivers, USB audio support, or amateur radio protocols, yet all of this code is compiled and loaded by default.

This unnecessary code termed *kernel bloat* carries measurable consequences. It enlarges the attack surface available to adversaries, increases memory usage, and prolongs system boot time. Gunasekaran et al. (2025) demonstrate that even after state-of-the-art automated debloating, 79–86% of kernel code remains unused at runtime, confirming that bloat is a persistent and largely unresolved problem.

Kernel debloating is the systematic process of identifying and removing unnecessary kernel code, retaining only what is required for target applications. The state-of-the-art automated debloating framework, Cozart (Kuo et al., 2020), achieves over 80% kernel size reduction through dynamic tracing of application workloads. Despite this, automated debloating remains rarely adopted in production. Cloud providers including Amazon AWS, Microsoft Azure, and Google Cloud continue to debloat kernels manually and ad hoc (Kuo et al., 2020), exposing their infrastructure to the very risks debloating is designed to address.

1.2. Motivation

The root cause of this limited adoption is an explicitly identified and completely unresolved safety limitation in all existing kernel debloating tools, stated in the doctoral dissertation behind Cozart (Kuo, 2022):

"The debloating techniques used by all studies in this thesis do not consider non-functional properties that could lead to heisenbugs. An example of a heisenbug could be a kernel module that initialises a variable used by another kernel module and this dependency is not specified."

A *heisenbug* is a defect that manifests unpredictably in production and disappears when examined making it extremely difficult to diagnose. In the debloating context it arises from **undocumented non-functional dependencies**: runtime relationships where one kernel module initialises shared state that another module silently depends upon, yet this relationship is not recorded anywhere in the kernel's KConfig configuration system and is invisible to dynamic tracing.

Gunasekaran et al. (2025) further confirm in 2025 that dynamic debloating tools capture what *executed* rather than what is *functionally essential* which is a fundamental limitation that directly enables this class of hidden dependency to persist undetected. No existing tool addresses this problem in the debloating context.

Importantly, the problem is not confined to any single debloating tool. The dependency exists in the kernel source itself: a configuration that disables the option gating the writer while keeping the option gating the reader produces the same uninitialised state, whether that configuration was produced by a debloating tool, by a distribution maintainer, or by hand. This research therefore studies the dependency as a property of the kernel, and treats debloating as the setting in which it becomes dangerous.

1.3. Revised Aims and Objectives

Research Aim: To determine whether static analysis of compiled kernel code can detect undocumented non-functional dependencies on global variables between Linux kernel compilation units, to demonstrate that such dependencies occur in configurations KConfig permits, and to show that a safety check built on this analysis can reject a configuration that would ship one.

Objectives:

O1. Extract, from the compiled objects of a configured kernel, every global variable together with the compilation units that write it and the compilation units in `net` that read it, covering both writes performed during `__init` (boot writers) and writes performed by ordinary code at run time (lazy writers).

O2. Filter these writer/reader pairs using the build conditions that gate each compilation unit, retaining only those pairs that a configuration could separate, that is, pairs where the writer can be removed while the reader and the variable's definition remain.

O3. Establish, for a candidate from O2, that the dependency is real by constructing a configuration KConfig permits, building it, and recording the observed runtime outcome.

O4. Classify the readers of the retained candidates by whether a test of the variable dominates its use, using a Clang/LLVM AST analysis applied to the shortlist produced by O2.

O5. Design a build-time safety check that consumes the output of O4 and rejects a removal set containing a writer whose reader is unguarded.

1.4. Changes Since the Proposal

The proposal defence raised three points, each of which has been addressed.

Table 1.1: Panel feedback and the resulting changes.

Feedback	**Change made**
A manually curated ground truth is not a sound basis for evaluation	The evaluation no longer rests on a dataset built by the author. It is restated as a precision and existence claim.
The scope is too broad	Reduced from two dependency types to one (writes to global variables, including function-pointer registrations), and from two subsystems to one (`net` readers).
The research question should not be framed around a single tool	The research questions now concern the dependency as a property of the kernel. Debloating is the motivating application, not the boundary of the question.

Chapter 2

Literature Review and Research Gap

2.1. Kernel Debloating: State of the Art

Configuration-based kernel tailoring was established by Kurmus et al. (2013), who reduced attack surface by deriving a kernel configuration from traced workloads, and its limits were examined by Alharthi et al. (2018), who showed that compile-time configuration alone leaves substantial unused code.

Cozart (Kuo et al., 2020, 2022) is the state-of-the-art automated framework — a *dynamic* tool that runs the kernel in QEMU, traces executed code, and assembles a minimised kernel achieving 80% size reduction with 13% boot time improvement. However, dynamic tracing creates a fundamental blind spot: it cannot observe dependencies between a removed module and a retained one, because the dependency only manifests when the removed module is absent.

More recent work addresses related but distinct problems. **SHARD** (Abubakar et al., 2021) enforces fine-grained control-flow integrity at runtime. **Hacksaw** (Hu et al., 2023) debloats based on hardware device inventory, but explicitly ignores indirect dependencies through shared data and function pointers to avoid false removals, precisely the class this research targets. Gunasekaran et al. (2025) improve over Cozart by up to 22%, yet confirm that 79–86% of kernel code remains unused post-debloating because dynamic traces capture *what executed*, not *what is functionally essential*. A 2024 comparative evaluation of ten debloating tools found only 13% produce sound programs (Brown et al., 2024); a systematic field review identifies dependency detection as an open challenge (Alhanahnah et al., 2024).

2.2. Configuration Dependencies and Static Analysis

The KConfig language is an unreliable dependency source. El-Sharkawy et al. (2015) analyse its semantics and show that its behaviour is sufficiently intricate that independent analysis tools disagree on the meaning of the same specification. Gunasekaran et al. (2025) likewise find many undocumented special cases, confirming that analysis of the C source and the build system is required rather than relying on configuration files alone.

Chen et al. (2020) demonstrate that static taint analysis can discover undocumented configuration parameter dependencies in Java systems with 87.9% recall and a 6.0% false positive rate, and this work is the methodological inspiration for the present research. However, cDep targets Java *parameter value* dependencies; this research targets C kernel *runtime state* dependencies, where the consequence of a missed dependency is uninitialised memory rather than a misconfigured value.

Static analysis of Linux kernel C code is well established in kernel development practice. Zhai et al. (2020) detect use-before-initialisation bugs using flow-sensitive analysis, validating static initialisation dependency detection in kernel C. Zhang et al. (2021) discover taint-style vulnerabilities in OS kernels at scale. Chen et al. (2024) demonstrate that automated dependency inference via global data container patterns uncovers dependencies overlooked by experts, directly supporting the feasibility of the approach taken here. Most critically, Hao et al. (2022) show that implicit dependencies from module initialisation through global memory account for 59–88% of uncovered kernel branches, and independently propose static analysis as the detection approach, confirming both the prevalence of the phenomenon and the appropriateness of the method.

2.3. Identified Research Gap

The Research Gap: Existing debloating tools reason about what a workload executed. Existing configuration tools reason about the dependencies that KConfig declares. Neither reasons about a writer whose option a configuration removes and a reader whose option it keeps, where no declared rule links the two. The author of Cozart identifies exactly this case as an unresolved limitation (Kuo, 2022), and the surrounding literature corroborates it: Hao et al. (2022) confirm that implicit dependencies from module initialisation are prevalent; Gunasekaran et al. (2025) confirm that dynamic tracing captures what executed rather than what is functionally essential; Brown et al. (2024) show that only 13% of debloating attempts across ten tools produce sound programs; and Alhanahnah et al. (2024) identify dependency identification as an open challenge for the field.

The gap is structural, not incidental. A dynamic approach can only observe dependencies between components that are both present during the trace. A dependency between a removed component and a retained one manifests only when the removed component is absent, which by definition does not occur during tracing. Only static analysis can reason about code that is not there.

2.4. The Dependency Pattern

The pattern this research targets has three parts: a global variable, a write to it from one compilation unit, and a read of it from another, where the build conditions of the two units differ and no KConfig rule links them.

Pattern 1 - Shared variable initialisation

```
/* mm/slab.c -- Module A */
void kmem_cache_init(void) {
    global_cache = kmem_cache_create(...);
}
/* fs/ext4/super.c -- Module B */
void ext4_init_cache(void) {
    ext4_inode_cachep = global_cache;   /* depends on Module A */
}
```

Module A writes the global variable `global_cache` during initialisation, and Module B reads it. If the option gating Module A is not enabled while the option gating Module B is, Module B copies a variable that was never set. The variable itself is defined in a unit other than Module A, so removing Module A does not remove the symbol. The build links successfully and the fault appears only at run time.

Pattern 2 - Function pointer registration

```
/* Unit A registers a handler into a shared pointer or ops table */
static int __init a_init(void) {
    kernel_ops = &a_handlers;
}
/* Unit B dispatches through it */
void b_work(void) {
    kernel_ops->do_something(data);   /* NULL if A is absent */
}
```

Unit B never names unit A. The dependency exists only through the shared pointer. This is the form most relevant to `net`, where subsystems commonly publish their entry points through registered operation tables, and it is related to the taint-style dependencies studied by Zhang et al. (2021).

Chapter 3

Research Questions, Significance and Scope

3.1. Revised Research Questions

RQ1. Can static analysis of compiled kernel objects, combined with the build conditions that gate each object, detect undocumented non-functional dependencies on global variables, where a configuration may remove the writer while retaining the reader?

RQ2. Can an AST-level guard analysis over the resulting candidates support a build-time safety check that rejects a configuration in which an unguarded reader is retained without its writer?

RQ1 concerns detection and is addressed by the work reported in Section 5. RQ2 concerns prevention; its design is given in Section 4 and its implementation is remaining work.

3.2. Justification of Significance

(a) Grounded in an admitted peer-reviewed gap. Kuo (2022) classifies this as an *essential* limitation, one that cannot be fixed by improving the implementation, and explicitly calls for future research to address it.

(b) Independently validated. Hao et al. (2022) confirm that implicit dependencies from module initialisation through global memory affect 59–88% of kernel branches. Brown et al. (2024) show only 13% of debloating attempts produce sound programs. Alhanahnah et al. (2024) identify this as an open field-wide challenge.

(c) Applicable beyond one tool. Because the dependency is a property of the kernel source and its build system, a detection method is applicable to any configuration process, not only to the traces published with one debloating framework.

(d) Tractable. The scope is deliberately narrowed to one dependency type, one subsystem and one kernel version, making the work completable within one academic year.

3.3. Scope and Delimitations

- **Kernel version:** Linux 4.18 (Ubuntu cosmic). This version is used because the configuration sets and build infrastructure available for this study target the 4.18 tree, and the method depends on the structure of the build system rather than on a particular release.
- **Dependency type:** writes to global variables, including function-pointer registrations. Both boot-time (`__init`) and run-time writers are covered.
- **Reader subsystem:** `net` only. Writers and variable definitions are drawn from `net`, `kernel`, `mm` and `lib`.
- **Analysis approach:** object-file analysis for candidate generation, Clang/LLVM AST analysis for guard classification on the resulting shortlist.

Table 3.1: Delimitations with justifications.

Delimitation	Justification
One dependency type only	Narrowed from the proposal on the panel's advice. Notification chains, mutex initialisation and initialisation-ordering dependencies require different analyses and are identified as future work.
One reader subsystem (`net`)	Full kernel analysis is impractical for one researcher in one year. `net` is a cohesive subsystem that makes heavy use of registered operation tables, the form most relevant to this dependency type.

Chapter 4

Research Approach and Methodology

4.1. Overview

The method has four stages. Stages 1 and 2 are implemented and have produced the results in Section 5. Stage 3 is designed and is the immediate next implementation step. Stage 4 is designed and is remaining work.

Table 4.1: Analysis stages and their status.

Stage	Function	Status
1	Extract global variables and their reference sites from compiled objects	Implemented
2	Filter writer/reader pairs by build condition	Implemented
3	Classify readers as guarded or unguarded (Clang AST)	Designed, not implemented
4	Build-time safety check consuming Stage 3 output	Designed, not implemented

4.2. Why Object-File Analysis Precedes Clang

The proposal specified Clang/LLVM AST analysis as the single analysis technique. During implementation this was revised into a two-stage design, for two reasons.

First, cost. A Clang pass over the full kernel requires per-file compilation flags and a successful parse of every translation unit under the target configuration, which is substantial work for a kernel of this age whose source is written for GCC. Object-file analysis requires only a completed build and standard binary utilities, and it processes the whole subsystem in minutes.

Second, fidelity. Object files describe the kernel that the configuration actually produced. The build system has already resolved which units are compiled, and the

relocations in each object record precisely which global each instruction references.

The limitation of object-file analysis is that it cannot reliably recognise a guard: a conditional test present in the source may be compiled into a register comparison that the binary-level view cannot attribute to the variable. Guard recognition is therefore deferred to Stage 3, where Clang examines the source of the small number of readers that survive Stage 2. The two stages are complementary: the cheap analysis reduces the population, and the expensive analysis is applied only where it is needed.

4.3. Stage 1 - Extraction

For every object file in the built subtree, the symbol table is read to record each defined global variable with its section and size, and the disassembly with relocations is read to record every instruction that references such a variable. Each reference is classified by the form of the instruction as a read, a write, a read-modify-write, an address-taken reference, or a reference from static data. The enclosing function and its ELF section are recorded, which distinguishes writers executing during boot (`.init.text`) from writers executing at run time. Source file and line are recovered from debugging information.

4.4. Stage 2 - Filtering

Candidate pairs are selected by the following filters, applied in order. The build condition of a compilation unit is computed by resolving the Kbuild Makefile hierarchy from the unit's directory upwards, so that conditions contributed by a parent directory and by membership of a composite module are combined.

Table 4.2: Filter stages.

Filter	Condition
F0	Global variables defined in the built set, excluding read-only data
F1	... written by at least one function of the selected kind (boot or run time)
F2	... and read by at least one function in `net`
F3	... where writer and reader are in different compilation units
F4	... where the variable is defined in a unit other than the writer's, so that removing the writer leaves the symbol defined and the kernel still links
F5	... where the build condition of the definition differs from that of the writer, so that some configuration can separate them

A point of interpretation applies throughout the table. Filters F0 to F2 count *variables*, because each asks a question about a variable as a whole. Filters F3 to F5 count *pairs*, because from F3 onward the question concerns a particular writer taken together with a particular reader. A variable with one writer and fifty readers contributes one unit to the F2 count and fifty to the F3 count. For this reason the pair totals are dominated by a small number of widely read variables, and the count of distinct variables surviving F5 is the figure that describes the size of the population requiring further examination.

4.4.1 F0 - The candidate population

F0 reads the symbol table of every object file and retains the entries that describe a defined variable. Entries in read-only sections are discarded, since a variable that cannot be written at run time cannot lose an initialisation. Function symbols and undefined references are likewise discarded. What remains is the set of variables that could in principle be left uninitialised.

4.4.2 F1 - Variables that are written

F1 asks whether any instruction writes the variable, and whether the writing function is of the kind under consideration. The access is taken from the form of the instruction recorded in Stage 1: a store or a read-modify-write qualifies, whereas a load or an instruction that merely takes the variable's address does not. The kind of writer is

determined by the ELF section of the enclosing function: a writer in `.init.text` runs once during boot, and a writer in any other executable section may run at any time.

The great majority of variables are removed here, because most global data in the kernel is established by its compile-time initialiser and never written afterwards. It is important to record what this removal does and does not mean. A variable removed by F1 has no *directly visible* writer. A variable written through a pointer, through a helper such as `memcpy`, through a sysctl handler, or by code outside the compiled subtrees is removed even though a writer exists. F1 therefore narrows the population; it does not certify the variables it discards.

4.4.3 F2 - Variables that are read in the subsystem of interest

F2 retains the variables that are also loaded by code in the reader subsystem, which for this work is `net`. The scope is applied here rather than during extraction, so that writers and definitions remain visible across all compiled subtrees while readers are restricted to the subsystem under study. A variable written in `net` but read only in `kernel` or `mm` is therefore removed at this filter, and is removed by the choice of scope rather than by any property of the variable.

4.4.4 F3 - Separable compilation units

F3 forms pairs and retains those whose writer and reader lie in different object files. An object file is compiled or omitted as a whole, so a writer and a reader within one object can never be separated by a configuration: either both are present or neither is. Pairs internal to a single object are consequently of no interest, although a dependency may still exist between two regions of one file that are governed by different preprocessor conditions. That case requires an analysis of preprocessor guards and lies outside the present pipeline.

4.4.5 F4 - The silent-failure filter

F4 is the filter that isolates the dangerous case, and it is the only one that examines the variable's definition rather than its uses. For each surviving pair it asks whether some definition of the variable lies in an object other than the writer's.

The reasoning is as follows. Suppose the variable were defined in the same object as its writer. Removing that object would remove the definition along with the writer, the reader's reference would become unresolved, and the link would fail. Such a configuration is rejected at build time, loudly and immediately, and it can never reach a running system. Now suppose instead that the definition lies elsewhere. Removing the writer's object leaves the variable in place, holding whatever value its compile-time initialiser

gave it. The reader still resolves, the kernel links without complaint, and the defect becomes observable only when the reader executes. The pairs surviving F4 are exactly those in the second situation, and that is the class this research concerns.

4.4.6 F5 - Differing build conditions

F5 retains a pair when the build condition of the definition differs from that of the writer. Identical conditions, including the case where both are unconditional, mean the two are always compiled together and no configuration can separate them.

The build condition of a compilation unit is not given by any single line of a Makefile. It is obtained by walking the Kbuild hierarchy upward from the unit's own directory, collecting every option that governs the unit along the way, and combining them conjunctively. Three mechanisms contribute. A unit may be named directly by an `obj-$(CONFIG_X)` assignment. It may instead be a member of a composite module, in which case both the member's own gate and the gate of the module it belongs to apply. Finally, the parent directory must itself be entered, which may be conditional. Resolving only the nearest mechanism yields an incomplete condition and, because F5 is a comparison between two such conditions, an incomplete condition can cause a pair to be retained or discarded wrongly.

4.4.7 What the filters establish, and what they do not

The filters establish that a pair is *structurally* separable: the writer and the reader occupy different units, the variable survives the writer's removal, and the two sides are governed by different options. Three further conditions must hold before a pair can be considered a genuine dependency, and none of them is tested by the pipeline as it stands.

(i) **Configuration reachability.** F5 compares the two conditions as text and does not capture Kconfig dependencies, so it cannot tell whether a configuration enabling the reader's option while disabling the writer's is actually permitted. Two approaches are available for later work: pose the question to a Kconfig solver, or check the pair against the composed configurations a debloating tool produces, since such a configuration is itself an instance of Kconfig's rules being satisfied. Neither is implemented.

(ii) **Loadability.** Even where Kconfig permits the combination, the reader's compilation unit may import other symbols that exist only when the writer's option is enabled. In that case the kernel fails to link, or the module fails to load, and the configuration never runs. This is a property of the symbols the two units exchange, not of their build conditions.

(iii) **Absence of a guard.** The reader may test the variable before use, in which case the kernel handles the uninitialised state deliberately. Stage 1 records whether a comparison instruction appears, but a source level check is frequently compiled into a form that this record cannot attribute to the variable. Guard classification is therefore the subject of Stage 3.

A pair that satisfies the filters but fails any of these three conditions is a structural separation that cannot occur, or that is handled when it does. The filters are accordingly best understood as a means of reducing a population of tens of thousands of variables to a few dozen distinct variables that merit the more expensive analyses, rather than as a test of whether a dependency is real.

4.5. Stage 3 - Guard Classification (designed)

For each reader surviving Stage 2, a Clang AST pass determines whether a test of the variable dominates its use, that is, whether every path from function entry to the dereference passes through a check. The analysis follows the value through a local variable when the reader assigns the global before use, and through inline accessor functions. Each reader is classified as guarded, unguarded, or unclear, and unclear cases are reported rather than silently resolved.

The classifier will be calibrated on readers whose status has first been established by inspecting the source, so that its verdicts can be compared against a known answer before it is relied upon.

4.6. Stage 4 - Build-Time Safety Check (designed)

The check consumes a proposed configuration together with the Stage 3 output and rejects the configuration when it removes a writer whose reader is retained and classified unguarded. The intended deployment is as a gate in an automated kernel build pipeline, placed after a configuration is produced and before the kernel is built.

Chapter 5

Experiments and Preliminary Results

5.1. Experimental Setup

All experiments use Linux 4.18 (Ubuntu cosmic, tree revision `66e4c0b79`) built with GCC 7.5.0 inside a container image fixed for the project. The subtrees `net`, `kernel`, `mm` and `lib` are compiled with debugging information enabled, producing 1,870 object files; a second build additionally including `fs` produces 3,026. Objects produced by linking several units together are excluded, since they duplicate the symbols of their members.

The analysis is reproducible. A complete re-run from a freshly configured tree produced byte-identical extraction tables and identical candidate tables, in the same order, to those produced by the original run. The recorded run captures the source revision, the configuration hash, the compiler version and the checksums of the analysis scripts.

5.2. Extraction and Filtering

Extraction over the 1,870 objects recorded 39,624 global variable definitions and 207,254 reference sites, with 14 instructions that the classifier could not assign to a category. Table 5.1 gives the filter results for `net` readers, separately for boot-time and run-time writers.

Table 5.1: Filter results for `net` readers. F0–F2 count variables; F3–F5 count writer/reader pairs.

Filter	Boot writers	Run-time writers
F0 global variable definitions	33,125	33,125
F1 written by a writer of this kind	212	315
F2 and read in `net`	76	117
F3 writer and reader in different units	4,103	3,561
F4 definition survives removal of the writer	1,817	2,764
F5 build conditions differ	1,739	2,610
Distinct variables surviving F5	34	42

The pair counts are dominated by a small number of widely read variables, so the distinct-variable count is the meaningful measure of the population a human or a subsequent analysis must examine. Taking boot and run-time writers together, and adding a separate run with `fs` readers, the union is 49 distinct variables. The reduction from 33,125 defined variables to 49 candidates is the contribution of Stage 2, and it is what makes the subsequent source-level analysis affordable.

5.3. Independence from a Particular Configuration

To confirm that the method does not depend on the configuration used for the build, the same analysis was repeated on a maximally enabled configuration (`allyesconfig`, 2,008 objects). This produced 2,657 surviving pairs over 34 distinct variables for boot writers and 1,959 pairs over 42 distinct variables for run-time writers, the same distinct-variable counts as the narrower build. Every distinct variable found in this wider build had already been surfaced by the narrower one, and a single additional variable appeared in the run-time set, which on inspection has a safe initial value and is only ever cleared, not initialised, by its writers.

This supports the position that the candidate set is a property of the kernel and its build system rather than of the particular configuration analysed.

5.4. Status of the Candidate Set

The candidates produced by Stage 2 have not yet been classified. Establishing whether a given reader guards the variable before using it requires the source-level analysis described as Stage 3, which is not yet implemented. The binary-level flag recorded during extraction is not a substitute: a check present in the source is frequently compiled into a form that cannot be attributed to the variable, so that flag is unreliable in both directions.

What the present work therefore establishes is the reduction itself. A population of tens of thousands of global variables is narrowed, by filters that are mechanical and reproducible, to a few dozen pairs whose writer and reader occupy different compilation units, whose variable survives removal of the writer, and whose two sides are governed by different build conditions. That set is small enough for the more expensive analyses to be applied to it, which is the purpose of the stage. Determining which members of it are genuine dependencies is the work of Stage 3 and the evaluation that follows.

5.5. Limitations of the Present Results

- **Guard recognition is not yet reliable.** The present pipeline flags the absence of a comparison instruction in the reader's compiled form, which does not correspond to the absence of a check in the source. Stage 3 exists to replace this.
- **Candidates are not yet classified.** No guard verdict is claimed for any candidate. A variable with several readers may have differing guard status across them, so classification must be per reader rather than per variable.
- **Recall is unmeasured.** The method cannot find dependencies carried by per-namespace structure fields, registration state, or initialisation ordering, and no measurement of what fraction of the true population is detected is claimed. See Section 6.

Chapter 6

Evaluation Plan

6.1. What Is Claimed, and What Is Not

The population of undocumented non-functional dependencies in the Linux kernel cannot be enumerated. No complete reference set exists, and constructing one by hand would make the author simultaneously the designer of the tool and the oracle against which it is judged. Recall is therefore not claimed and not measured.

The evaluation instead makes a **precision and existence** claim: of the candidates the method reports, what proportion are confirmed by a procedure fixed in advance. This is the position taken by kernel bug-finding research generally, where confirmed defects and precision are reported and completeness is disclaimed.

6.2. Confirmation Procedure

The criterion for confirming a candidate is fixed before the candidate set is examined, so that it cannot be adjusted to suit the output. A candidate is confirmed when all of the following hold:

C1. the call graph contains a path from a kernel entry point to the reader that does not require the writer's initialisation path to have executed;

C2. the Stage 3 guard analysis finds no test of the variable dominating the read;

C3. a KConfig solver reports that a configuration enabling the reader's option while disabling the writer's option is satisfiable.

The denominator is fixed in the same manner. Every candidate in the shortlist is carried through the procedure, and the outcome for each is reported as confirmed, refuted, or inconclusive. Reporting the inconclusive cases is part of the method, not an omission from it.

6.3. Evidence, Ranked by Independence

Table 6.1: Forms of confirming evidence, strongest first.

Form	Description
Runtime witness	A kernel built in a configuration where the writer's option is absent, booted in an isolated virtual machine, with the execution path leading to the reader exercised and the resulting kernel log recorded. The outcome is mechanical: the fault either occurs or it does not.
Mechanical static confirmation	Criteria C1–C3, each computed by a tool rather than asserted. To prevent the reachability analysis from inheriting the detector's blind spots, the call graph is produced using an established external tool rather than code written for this project.
Source examination	Used only where the above cannot be applied, and reported as such.

The runtime witness is the decisive evidence for readers classified unguarded. A single observed fault establishes that the dependency class occurs in practice, in a configuration KConfig permits, independently of any judgement by the author.

6.4. Capability Check on Known Instances

Instances of this dependency class that were corrected during the kernel's development history are used as a capability check: the detector is run against the source as it stood before each correction, and it is recorded whether the instance is flagged. This measures whether the method is able to find instances of the shape in question. It is not a recall measurement, because the set of historical instances is itself incomplete and was not sampled from a known population.

6.5. Comparison with Existing Analyses

The claim that existing tools do not detect this dependency class is at present an argument from their design. It will be supported by running established kernel analyses,

including Smatch and the Clang Static Analyzer, over the same instances used in the capability check, and recording what each reports. A baseline that does detect some instances is itself a useful finding and will be reported as such.

6.6. Functional Check on the Retained Kernel

After removing writers that the method does not flag, the resulting kernel is exercised with the `net` subsystem's self-tests and, where available, the LTP networking tests, in the manner Cozart applies to its own output. This demonstrates that the retained kernel remains functional. It is explicitly not evidence about the dependencies themselves: Kuo et al. (2020) report that test suites cover only a small proportion of kernel code under coverage measurement, so a reader that is guarded, or that is never reached by the tests, may still matter in configurations the tests do not exercise. Guarded candidates are therefore retained in the shortlist rather than discarded, and the question of whether a guarded dependence is genuinely required is left to later work.

6.7. Evaluation of RQ2

The safety check is evaluated by applying it to removal sets constructed to contain a writer whose reader is confirmed unguarded, and to matched sets whose readers are guarded, recording whether it rejects the former and accepts the latter.

6.8. Threats to Validity

- Recall is unmeasured.
- Results are obtained from one kernel version and one subsystem and may not generalise to others.
- The capability check uses historical instances that are an incomplete and possibly unrepresentative sample of the dependency class.

Chapter 7

Availability of Required Resources

All resources required are publicly available at no cost. No proprietary data, human participants, or ethical approval are required. The resources listed in the proposal remain accurate; those in current use are summarised below.

Table 7.1: Resources in use.

Resource	**Source**	**Purpose**
Linux kernel source (4.18)	kernel.org, Ubuntu — free	Analysis target
Cozart framework and configurations	GitHub — open source (Kuo et al., 2020)	Configuration sets for the debloating application
GNU binutils, GCC	Open source — free	Building the kernel and reading compiled objects
Clang/LLVM	Open source — free	Guard classification (Stage 3)
kconfiglib	Open source — free	Evaluating whether a configuration permits a separation
Smatch, Clang Static Analyzer	Open source — free	Baselines for comparison
QEMU virtualisation	Open source — free	Isolated kernel execution for the runtime witness

Hardware: A standard machine with 16 GB RAM and 8 CPU cores is sufficient for kernel compilation and virtualised testing.

Chapter 8

Project Progress and Remaining Plan

8.1. Project Schedule

Figure 8.1 shows the schedule across the twelve-month project. The dashed line marks the end of the interim report. Dark bars show work completed up to that point, and light bars show work planned after it.

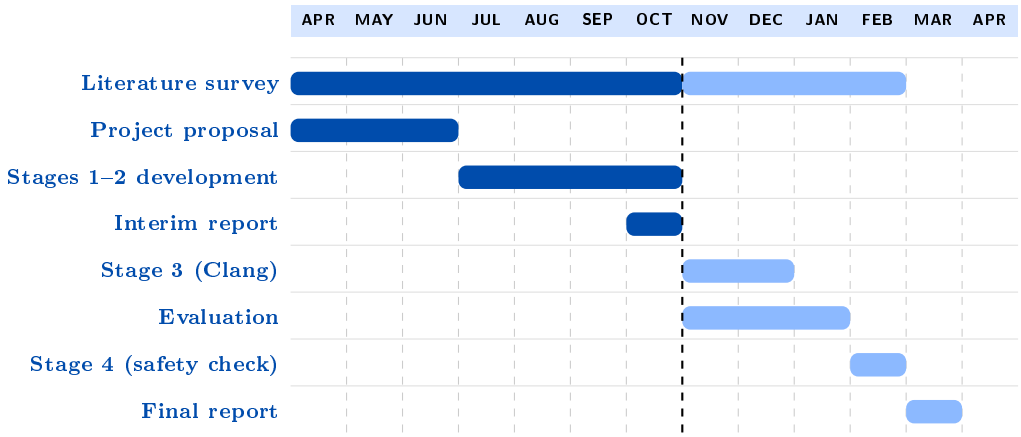


Figure 8.1: Project schedule. Dashed line: end of the interim report. Dark bars: work completed up to the interim report. Light bars: planned work.

8.2. Progress to Date

Table 8.1: Work completed.

Item	Outcome
Extraction (Stage 1)	Implemented and validated on synthetic inputs before use; processes 1,870 objects, recording 39,624 variables and 207,254 reference sites.
Filtering (Stage 2)	Implemented, including resolution of build conditions through the Makefile hierarchy, including the composite-object and text-substitution forms; reduces 33,125 defined variables to 49 candidates.
Reproducibility	Full re-run from a fresh build reproduces extraction and candidate tables identically; run metadata recorded.
Candidate set	Produced and recorded, but not yet classified. Guard status per reader requires Stage 3, which is not implemented.

8.3. Remaining Plan

Table 8.2: Remaining work.

Period	Activity	Milestone
Months 6–7	Implement Stage 3 guard classification; fix confirmation criteria and denominator in writing	Guard status for every reader of every candidate
Months 8–9	Call graph with an external tool; KConfig satisfiability for each candidate; runtime witness for unguarded readers	Confirmed, refuted and inconclusive counts over a fixed candidate set
Month 10	Capability check on historical instances; baseline comparison against Smatch and the Clang Static Analyzer	Evidence for the claim that existing tools do not report this class
Month 11	Stage 4 safety check	Check demonstrated to reject unsafe and accept safe removal sets
Month 12	Write-up and submission	Final report

8.4. Risk Assessment

Table 8.3: Project risks and mitigation strategies.

Risk	Likelihood	Mitigation
Guard classification proves inaccurate	Medium	Calibrated against readers whose status has been established by source inspection; unclear cases reported rather than resolved silently
Runtime witness does not reproduce the fault	Medium	A negative result is reported as such and refutes that candidate under the conditions tested; the static analysis result stands independently
Few candidates confirmed	Low–Medium	Precision is the reported quantity; a low confirmed count with a documented procedure is a valid result that answers the question asked

References

Abubakar, M., Ahmad, A., Fonseca, P., and Xu, D. (2021). SHARD: Fine-grained kernel specialization with context-aware hardening. In *Proceedings of the 30th USENIX Security Symposium (USENIX Security 2021)*, pages 2435–2452, Virtual. USENIX Association.

Alhanahnah, M., Boshmaf, Y., Gehani, A., Craven, R., and Mickelson, M. (2024). SoK: Software debloating landscape and future directions. In *Proceedings of the 2024 Workshop on Forming an Ecosystem Around Software Transformation (FEAST@CCS 2024)*, pages 11–18, Salt Lake City, UT, USA. ACM.

Alharthi, M., Hu, H., Moon, H., and Kim, T. (2018). On the effectiveness of kernel debloating via compile-time configuration. In *Proceedings of the 1st Workshop on Software Debloating and Delayering (FEAST 2018)*, Amsterdam, Netherlands. ACM.

Brown, M. D., Meily, A., Fairservice, B., Sood, A., Dorn, J., Kilmer, E., and Eytchison, R. (2024). A broad comparative evaluation of software debloating tools. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 2024)*, pages 3927–3943, Philadelphia, PA, USA. USENIX Association.

Chen, Q., Wang, T., Legunsen, O., Li, S., and Xu, T. (2020). Understanding and discovering software configuration dependencies in cloud and datacenter systems. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*, pages 362–374, New York, NY, USA. ACM.

Chen, W., Hao, Y., Zhang, Z., Zou, X., Kirat, D., Mishra, S., Schales, D., Jang, J., and Qian, Z. (2024). SyzGen++: Dependency inference for augmenting kernel driver fuzzing. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (S&P 2024)*, pages 4661–4677, San Francisco, CA, USA. IEEE.

El-Sharkawy, S., Krafczyk, A., and Schmid, K. (2015). Analysing the Kconfig semantics and its analysis tools. In *Proceedings of the 14th International Conference on Generative Programming: Concepts and Experiences (GPCE 2015)*, pages 45–54, Pittsburgh, PA, USA. ACM.

Gunasekaran, A., Ritter, G., and Bobba, R. B. (2025). In pursuit of lean OS kernels: Improving configuration-based debloating. In *Proceedings of the 2025 Annual Com-*

puter Security Applications Conference (ACSAC 2025), pages 154–167, Austin, TX, USA. IEEE.

Hao, Y., Zhang, H., Li, G., Du, X., Qian, Z., and Sani, A. A. (2022). Demystifying the dependency challenge in kernel fuzzing. In *Proceedings of the 44th International Conference on Software Engineering (ICSE 2022)*, pages 659–671, Pittsburgh, PA, USA. ACM.

Hu, Z., Lee, S., and Peinado, M. (2023). Hacksaw: Hardware-centric kernel debloating via device inventory and dependency analysis. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, pages 1994–2008, Copenhagen, Denmark. ACM.

Kuo, H.-C. (2022). *Attack Surface Reduction in Contemporary Operating Systems via Practical Kernel Debloating*. PhD dissertation, University of Illinois Urbana-Champaign, Urbana, Illinois, USA.

Kuo, H.-C., Chen, J., Mohan, S., and Xu, T. (2020). Set the configuration for the heart of the OS: On the practicality of operating system kernel debloating. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 4(1):1–27.

Kuo, H.-C., Chen, J., Mohan, S., and Xu, T. (2022). Set the configuration for the heart of the OS: On the practicality of operating system kernel debloating. *Communications of the ACM*, 65(5):101–109.

Kurmus, A., Tartler, R., Dorneanu, D., Heinloth, B., Rothberg, V., Ruprecht, A., Schroeder-Preikschat, W., Lohmann, D., and Kapitza, R. (2013). Attack surface metrics and automated compile-time OS kernel tailoring. In *Proceedings of the 20th Annual Network and Distributed System Security Symposium (NDSS 2013)*, San Diego, CA, USA. Internet Society.

Zhai, Y., Hao, Y., Zhang, H., Wang, D., Song, C., Qian, Z., Lesani, M., Krishnamurthy, S. V., and Yu, P. (2020). UBITect: A precise and scalable method to detect use-before-initialization bugs in Linux kernel. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*, pages 1303–1314, Virtual Event, USA. ACM.

Zhang, H., Chen, W., Hao, Y., Li, G., Zhai, Y., Zou, X., and Qian, Z. (2021). Statically discovering high-order taint style vulnerabilities in OS kernels. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS 2021)*, pages 811–824, Virtual Event, Republic of Korea. ACM.